



UNIVERSITY OF BUEA

Faculty of Engineering and Technology

CEF 440

Internet Programming and Mobile Programming

Group 16

TASK 7: FINAL IMPLEMENTATION

Submitted to:

Dr. Nkemeni Valery

2025/2026

GROUP MEMBERS

Name	Matricule
KEUKANG CARMEN ALICIA	FE23A074
NKENGAFAC WILLYHERMAN FONKEM	FE23A119
NKENG SAMA MOKOM	FE23A118
CHIANGOIESEH CHRIS RAWLINGS	FE23A029
ENOW JOHN EYONG	FE23A046

Table of Contents

CHAPTER ONE	6
1.1 Introduction.....	6
1.2 Background of the Problem	6
1.3 Problem Statement	7
1.4 Aim of the Project.....	8
1.5 Objectives of the Project	8
1.6 Scope of the Project	8
1.7 Significance of the Study	9
1.8 Structure of the Report.....	Error! Bookmark not defined.
1.9 Chapter Summary	9
CHAPTER TWO	10
2.1 Introduction.....	10
2.2 Quality of Service (QoS).....	10
2.2.1 Latency.....	10
2.2.2 Throughput.....	10
2.2.3 Packet Loss	10
2.2.4 Jitter.....	11
2.2.5 Signal Strength.....	11
2.3 Quality of Experience (QoE)	11
2.4 Relationship Between QoS and QoE	12
2.5 Review of Existing Network Monitoring Applications	12
2.5.1 Speedtest	12
2.5.2 nPerf.....	12
2.5.3 OpenSignal.....	13
2.6 Gap Analysis.....	13
2.7 Relevant ITU Standards	13
2.8 Summary of the Literature Review	Error! Bookmark not defined.
CHAPTER THREE	15
3.1 Introduction.....	15
3.2 Stakeholder Identification	15
3.2.1 Mobile Network Users	15
3.2.2 Mobile Network Operators	15
3.2.3 System Administrator	15
3.2.4 Developers	16

3.3 Requirement Gathering Techniques.....	16
3.3.1 Surveys.....	16
3.3.2 Brainstorming	17
3.3.3 Reverse Engineering	17
3.3.5 Document Analysis	17
3.4 Data Gathering Results	17
3.5 Data Cleaning.....	19
3.6 User Reluctance Assessment	19
3.7 Challenges Encountered During Requirement Gathering.....	20
3.8 Chapter Summary	Error! Bookmark not defined.
CHAPTER FOUR.....	21
4.1 Introduction.....	21
4.4 Non-Functional Requirements	23
4.5 Requirement Prioritisation Using the MoSCoW Technique.....	24
4.6 Software Requirements Specification (SRS)	25
4.6.1 Purpose.....	26
4.6.2 Intended Users	26
4.6.3 System Overview	26
4.6.4 Functional Specification	26
4.6.5 External Interfaces	27
4.6.6 Constraints	27
4.6.7 Assumptions.....	28
4.7 Requirement Traceability.....	28
4.8 Chapter Summary	Error! Bookmark not defined.
CHAPTER FIVE	29
5.1 Introduction.....	29
5.2 System Architecture.....	29
5.3 Context Diagram.....	31
5.4 Data Flow Diagram (DFD)	31
5.4.1 Level 0 Data Flow Diagram.....	32
5.4.2 Level 1 Data Flow Diagram.....	33
5.5 Use Case Diagram.....	34
5.6 Sequence Diagrams.....	35
5.7 Class Diagram.....	36
5.8 Deployment Diagram.....	37

5.9 Design Considerations	38
5.10 Chapter Summary	Error! Bookmark not defined.
CHAPTER SIX	39
6.1 Introduction.....	39
6.2 Application Identity	39
6.2.1 Application Logo	39
6.2.2 Colour Scheme.....	40
6.2.3 Typography	40
6.3 User Interface Design	40
6.3.1 Figma Design Mockups	41
6.4 Frontend Implementation.....	44
6.5 Description of Implemented Screens	44
6.6 Challenges Encountered During Implementation	45
6.9 Chapter Summary	Error! Bookmark not defined.
CHAPTER SEVEN	47
7.1 Introduction.....	47
7.2 Data Elements	47
7.3 Conceptual Database Design	48
7.4 Entity Relationship Diagram.....	49
7.5 Database Implementation.....	49
7.6 Mongoose Schemas	50
7.7 Backend Implementation	51
7.8 API Endpoints.....	52
7.9 Connecting the Database to the Backend and Frontend.....	53
7.10 Challenges Encountered.....	54
7.11 Chapter Summary	Error! Bookmark not defined.
CHAPTER EIGHT	55
8.1 Introduction.....	55
8.2 Testing Strategy	55
8.3 Test Environment.....	55

CHAPTER ONE

INTRODUCTION

1.1 Introduction

The rapid growth of mobile communication technologies has transformed the way individuals and organizations communicate, access information, and conduct business. Mobile networks have become an essential component of modern society, providing services such as voice communication, internet access, online banking, e-learning, telemedicine, and social networking. As smartphone adoption continues to increase across developing countries, users expect reliable network connectivity and satisfactory internet performance regardless of their location.

In Cameroon, the telecommunications sector has experienced significant expansion over the past decade. Mobile Network Operators (MNOs) such as MTN Cameroon, Orange Cameroon, and Camtel have invested considerably in expanding their network infrastructure to support the growing number of subscribers. Despite these improvements, many users continue to experience inconsistent network performance characterized by poor signal strength, slow internet speeds, high latency, frequent call drops, and unstable connectivity. These issues negatively affect user satisfaction and limit the effectiveness of digital services that rely on stable internet connections.

Quality of Service (QoS) and Quality of Experience (QoE) are two important concepts used to evaluate mobile network performance. While QoS focuses on measurable technical parameters such as bandwidth, latency, packet loss, and throughput, QoE evaluates the overall satisfaction of end users when interacting with network services. Both metrics are essential for understanding the actual performance delivered by mobile network providers.

To address these challenges, this project presents NetInsight, an Android mobile application developed using React Native and Expo. The application enables users to measure and monitor key network performance indicators while providing an intuitive interface for viewing collected network information. The system uses an Express.js backend connected to MongoDB Atlas for storing and managing collected network performance data. By combining technical network measurements with user feedback, NetInsight aims to provide valuable insights into mobile network performance in Cameroon.

This chapter introduces the background of the study, states the research problem, presents the aim and objectives of the project, defines its scope, and outlines the overall structure of the report.

1.2 Background of the Problem

The increasing dependence on mobile internet services has made network quality a critical concern for both consumers and service providers. Mobile applications such as online learning platforms, cloud storage systems, streaming services, video conferencing tools, and financial applications require reliable internet connectivity to function effectively. Consequently, users

expect mobile networks to provide consistent and high-quality service regardless of their geographical location.

Although Cameroon has witnessed considerable improvements in telecommunications infrastructure, network performance remains inconsistent in many regions. Urban centers generally enjoy better connectivity than rural communities, where limited infrastructure often leads to poor network coverage and reduced internet quality. Even within cities, network congestion during peak hours frequently causes slow internet speeds and increased latency.

Most users rely on subjective experiences to determine the quality of their network connection. When internet performance becomes poor, users often struggle to determine whether the issue originates from their device, their location, or the network provider. Furthermore, available information regarding network quality is usually limited to isolated speed tests that do not provide continuous monitoring or comprehensive analysis.

Existing network testing applications provide useful measurements but often prioritize raw technical metrics without integrating localized user experience data relevant to Cameroon. Consequently, users and researchers lack accessible tools that combine objective network measurements with practical user-oriented analysis.

Developing a localized network monitoring application capable of collecting network performance data and presenting meaningful insights can contribute toward improved understanding of mobile network quality while assisting users in making informed decisions regarding their connectivity.

1.3 Problem Statement

Despite continuous investment by telecommunications providers, many mobile users in Cameroon continue to experience inconsistent internet connectivity, poor download and upload speeds, unstable signal strength, and high network latency. These issues directly affect productivity, communication, online education, entertainment, and access to digital services.

Although several commercial applications exist for measuring internet performance, many focus primarily on individual speed tests without offering a comprehensive platform that combines Quality of Service measurements, Quality of Experience evaluation, and centralized storage of historical performance data. Additionally, most available solutions are designed for global audiences and do not specifically address the needs and network conditions experienced by users in Cameroon.

Therefore, there is a need for an accessible mobile application that enables users to evaluate network performance, monitor important QoS metrics, and provide user feedback to support QoE assessment while storing collected information for future analysis.

1.4 Aim of the Project

The aim of this project is to design and develop NetInsight, an Android-based mobile application for monitoring and evaluating mobile network Quality of Service (QoS) and Quality of Experience (QoE) in Cameroon.

1.5 Objectives of the Project

The specific objectives of this project are:

- To study existing approaches used for monitoring mobile network performance.
- To identify the requirements of users regarding network quality assessment.
- To design an intuitive mobile application for monitoring network performance.
- To develop the frontend of the application using React Native and Expo.
- To implement a backend system using Express.js.
- To design and implement a MongoDB Atlas database for storing network performance records.
- To integrate the frontend, backend, and database into a functional application.
- To evaluate the application's ability to monitor and present network performance information.

1.6 Scope of the Project

This project focuses on the development of an Android mobile application that enables users to monitor mobile network performance using selected Quality of Service metrics while supporting user feedback for evaluating Quality of Experience.

The scope of the project includes:

- Development of an Android application using React Native and Expo.
- Collection of selected network performance metrics.
- Storage of collected network information in MongoDB Atlas.
- Backend API development using Express.js.
- Display of network performance information through an intuitive user interface.

The project does not include:

- Development of an iOS version.
- Direct integration with mobile network operators' internal infrastructure.
- Real-time nationwide network optimization.

- Advanced machine learning models for predictive network analysis.
- Automatic modification or improvement of network performance.

1.7 Significance of the Study

This project is expected to benefit several stakeholders.

Users will gain a better understanding of the quality of their mobile network through objective performance measurements and historical records. Network researchers may use collected data to study connectivity patterns and identify areas requiring improvement. Mobile network operators may also benefit from aggregated performance information that highlights regions experiencing poor service quality. Finally, the project provides valuable practical experience in full-stack mobile application development involving React Native, Express.js, MongoDB Atlas, RESTful APIs, and modern software engineering practices.

1.8 Chapter Summary

This chapter introduced the NetInsight project by discussing the background of mobile network quality challenges in Cameroon, identifying the research problem, defining the project aim and objectives, outlining the scope of the study, highlighting its significance, and presenting the structure of the report. The next chapter reviews existing literature related to Quality of Service, Quality of Experience, current network monitoring applications, relevant standards, and research gaps that motivate the development of NetInsight.

CHAPTER TWO

LITERATURE REVIEW

2.1 Introduction

A literature review provides the theoretical and practical foundation upon which a research project is built. It examines previous studies, existing technologies, and current solutions related to the problem being addressed. For this project, the literature review focuses on mobile network Quality of Service (QoS), Quality of Experience (QoE), existing network monitoring applications, identified research gaps, and relevant international standards. Understanding these concepts helps justify the development of NetInsight and demonstrates how the proposed system contributes to improving mobile network performance monitoring in Cameroon.

2.2 Quality of Service (QoS)

Quality of Service (QoS) refers to the measurable performance characteristics of a communication network. It is concerned with the network's ability to deliver data efficiently while meeting predefined performance requirements. Unlike subjective user opinions, QoS is evaluated using objective technical metrics that describe the behavior of the network.

Telecommunication operators monitor QoS to ensure that their networks provide reliable communication services. Several parameters are commonly used to assess QoS.

2.2.1 Latency

Latency is the time taken for a data packet to travel from the source device to the destination and back again. It is typically measured in milliseconds (ms). Low latency is essential for applications such as online gaming, video conferencing, voice-over-IP (VoIP), and live streaming, where delays can significantly affect user experience.

2.2.2 Throughput

Throughput represents the amount of data successfully transmitted over a network within a specified period. It is commonly measured in megabits per second (Mbps). Higher throughput generally indicates better network performance, particularly when downloading or uploading large files.

2.2.3 Packet Loss

Packet loss occurs when transmitted data packets fail to reach their intended destination. High packet loss can result in interrupted voice calls, buffering during video streaming, and incomplete data transmission.

2.2.4 Jitter

Jitter measures the variation in packet arrival times. Even when average latency is low, high jitter can negatively affect real-time communication applications by causing audio distortion and video interruptions.

2.2.5 Signal Strength

Signal strength indicates how effectively a mobile device communicates with a nearby cellular base station. Weak signal strength often results in slower internet speeds, dropped calls, and unstable connectivity.

These QoS parameters collectively provide an objective assessment of network performance and are commonly used by telecommunication providers to evaluate service quality.

2.3 Quality of Experience (QoE)

While QoS focuses on technical measurements, Quality of Experience (QoE) evaluates how users perceive the quality of the services they receive. QoE considers the satisfaction of end users based on their interaction with network services.

QoE depends not only on network performance but also on user expectations, application requirements, device capabilities, and environmental conditions. For example, two users connected to the same network may report different experiences depending on the applications they are using.

Common factors influencing QoE include:

- Page loading speed.
- Video streaming quality.
- Voice call clarity.
- Reliability of internet connectivity.
- Frequency of connection interruptions.
- Overall user satisfaction.

Modern network evaluation increasingly combines QoS measurements with QoE assessments because technical performance alone does not always reflect the actual experience of users.

2.4 Relationship Between QoS and QoE

Although QoS and QoE are closely related, they measure different aspects of network performance.

QoS evaluates technical characteristics that can be measured objectively using network tools and protocols. In contrast, QoE measures the subjective perception of users while using network services.

For example, a network may record acceptable download speeds and latency while users still experience poor video quality due to application-specific issues or network instability. Similarly, slight reductions in throughput may not significantly affect users browsing simple websites but may greatly impact video streaming or online gaming.

Therefore, an effective network monitoring solution should consider both QoS and QoE to provide a comprehensive evaluation of network performance. This principle forms the foundation of the NetInsight application.

2.5 Review of Existing Network Monitoring Applications

Several mobile applications have been developed to measure internet performance and network quality. These applications provide useful benchmarking tools for evaluating connectivity. The following section reviews three of the most widely used applications.

2.5.1 Speedtest

Speedtest is one of the most popular internet performance testing applications. It measures download speed, upload speed, latency, and jitter by connecting users to nearby servers.

The application provides quick performance results and maintains a history of previous tests. It also allows users to compare internet performance across different locations.

Despite its popularity, Speedtest primarily focuses on isolated speed measurements and offers limited integration of user experience feedback and no continuous background monitoring.

2.5.2 nPerf

nPerf extends traditional speed testing by including additional performance evaluations such as web browsing tests, streaming quality assessment, and network coverage mapping.

Its comprehensive testing capabilities make it useful for evaluating multiple aspects of internet performance. However, many advanced analytical features require internet connectivity and may not specifically address localized network conditions in developing countries.

2.5.3 OpenSignal

OpenSignal collects crowd-sourced network measurements from users around the world. It provides coverage maps, network availability statistics, download speeds, and operator comparisons.

One of OpenSignal's strengths is its ability to visualize network performance geographically. However, much of its analysis depends on the availability of large-scale user contributions, and localized customization for specific regions may be limited.

2.6 Gap Analysis

Although existing network monitoring applications provide valuable functionality, several limitations remain.

First, most applications prioritize technical QoS measurements while providing limited opportunities for users to express their actual satisfaction with network performance. Consequently, important aspects of Quality of Experience may not be adequately represented.

Second, many commercial applications are designed for global audiences and may not address localized network conditions experienced in Cameroon. Factors such as inconsistent infrastructure, regional coverage differences, and varying service quality require solutions adapted to local environments.

Third, some existing applications store only limited historical information accessible to individual users without providing an integrated backend architecture suitable for future analytical extensions.

NetInsight addresses these gaps by combining QoS measurements with user-oriented QoE evaluation, and continuous background monitoring while maintaining centralized storage through MongoDB Atlas. The use of a React Native frontend and Express.js backend also provides a flexible architecture for future enhancements.

2.7 Relevant ITU Standards

The International Telecommunication Union (ITU) develops globally recognized standards for evaluating communication networks and user experience. These recommendations provide guidance for measuring network performance and ensuring consistency across telecommunications systems.

Several ITU recommendations are relevant to this project.

ITU-T Recommendation G.1010

This recommendation defines Quality of Service requirements for different multimedia applications, including voice communication, video conferencing, and internet services. It specifies acceptable performance levels for latency, jitter, and packet loss.

ITU-T Recommendation P.10

This recommendation provides standardized terminology related to Quality of Experience. It establishes definitions that promote consistency when evaluating user satisfaction across different communication services.

ITU-T Recommendation E.800

Recommendation E.800 defines concepts and terminology associated with Quality of Service in telecommunications. It explains the relationship between service quality from both technical and user perspectives and serves as one of the fundamental references for QoS evaluation.

These standards provide internationally accepted guidelines that support the design and evaluation of network performance monitoring systems such as NetInsight.

CHAPTER THREE

REQUIREMENT GATHERING

3.1 Introduction

Requirement gathering is one of the most critical phases of the Software Development Life Cycle (SDLC). It involves identifying the needs and expectations of stakeholders to ensure that the final system effectively solves the intended problem. Accurate requirement gathering minimizes the risk of developing features that do not meet user needs and provides a clear foundation for system analysis, design, and implementation.

For the NetInsight project, several requirement elicitation techniques were employed to understand the challenges faced by mobile network users in Cameroon and to determine the features that would be most beneficial in a mobile network monitoring application. This chapter discusses the stakeholders involved, the methods used to gather requirements, the collected data, data cleaning procedures, and the assessment of user participation and reluctance during the requirement gathering process.

3.2 Stakeholder Identification

Stakeholders are individuals or organizations that influence or are affected by the development and use of a software system. Identifying stakeholders ensures that different perspectives are considered during system development.

The primary stakeholders for the NetInsight project include:

3.2.1 Mobile Network Users

Mobile network users are the primary users of the application. They require a simple and reliable platform for evaluating internet performance, monitoring network quality, and viewing historical test results. Their feedback was essential in determining the application's functional requirements.

3.2.2 Mobile Network Operators

Although mobile network operators do not directly interact with the application, they represent important secondary stakeholders. Aggregated network performance information can help identify areas experiencing poor service quality and support future infrastructure improvements.

3.2.3 System Administrator

The system administrator is responsible for maintaining the backend services and database. Responsibilities include monitoring stored data, ensuring system availability, managing user information, and maintaining API functionality.

3.2.4 Developers

Developers are responsible for designing, implementing, testing, and maintaining the application. They use the gathered requirements to guide software architecture, interface design, and implementation decisions.

3.3 Requirement Gathering Techniques

To obtain comprehensive and reliable requirements, multiple requirement elicitation techniques were employed.

3.3.1 Surveys

Questionnaires were distributed to smartphone users to collect quantitative information regarding their experiences with mobile network performance. The survey included questions related to internet speed, network reliability, frequency of connectivity issues, and desired application features.

The survey enabled data collection from a relatively large number of respondents within a short period while ensuring consistency across responses.

Figure 3.1: Survey Questions

1	12. Overall, how satisfied are you with your m	13. How often do you experience poor netwo	14. Which of these issues do you experience	15. At what time of day is your network usual	16. In which locations do you experience the	17. Have you ever missed something im
2		2 Daily	Internet being very slow / buffering	No specific time / Network is consistently poor	At home	Yes, many times
3		2 Daily	Internet being very slow / buffering, Unable to sen	No specific time / Network is consistently poor	At home, At school or workplace	Yes, once or twice
4		3 Daily	Internet being very slow / buffering	Afternoon (12 PM – 5 PM)	At school or workplace, In busy areas (markets, ci	Yes, once or twice
5		3 A few times a week	Internet being very slow / buffering	No specific time / Network is consistently poor	On public transport / traveling	Yes, once or twice
6		1 Daily	Internet being very slow / buffering	Morning (6 AM – 12 PM)	At home, At school or workplace	Yes, many times
7		2 Daily	Calls dropping suddenly, Internet being very slow	Night (10 PM – 6 AM)	At home	Yes, many times
8		3 A few times a week	Calls dropping suddenly, Internet being very slow	Evening (5 PM – 10 PM)	At home	Yes, once or twice
9		1 Daily	Internet being very slow / buffering	No specific time / Network is consistently poor	At school or workplace	Yes, many times
10		2 Daily	Internet being very slow / buffering	No specific time / Network is consistently poor	At home, At school or workplace, On public transp	Yes, once or twice
11		1 Daily	Internet being very slow / buffering, Unable to sen	No specific time / Network is consistently poor	At home, At school or workplace, In busy areas (n	Yes, many times
12		2 A few times a week	Calls dropping suddenly, Internet being very slow	No specific time / Network is consistently poor	At home	Yes, once or twice
13		3 A few times a week	Internet being very slow / buffering	Evening (5 PM – 10 PM)	At home	Yes, once or twice
14		3 A few times a week	Internet being very slow / buffering	Evening (5 PM – 10 PM)	At home	Yes, once or twice
15		1 Daily	Calls dropping suddenly, Internet being very slow	No specific time / Network is consistently poor	At home, On public transport / traveling	Yes, many times
16		3 Daily	Calls dropping suddenly, Internet being very slow	No specific time / Network is consistently poor	At home, In rural/remote areas	Yes, many times
17		2 Daily	Internet being very slow / buffering, Unable to sen	No specific time / Network is consistently poor	At home, At school or workplace, On public transp	Yes, once or twice
18		3 Daily	Internet being very slow / buffering	Evening (5 PM – 10 PM)	At home	Yes, once or twice

3.3.2 Brainstorming

Brainstorming sessions were carried out during the planning stage to identify possible system features and technical solutions. Ideas generated during these sessions were evaluated based on feasibility, usefulness, implementation complexity, and project constraints.

This process contributed to defining the overall functionality of NetInsight.

3.3.3 Reverse Engineering

Existing network monitoring applications such as Speedtest, nPerf, and OpenSignal were analyzed to understand their functionality, interface design, and feature sets.

Reverse engineering was not intended to duplicate these applications but rather to identify best practices and opportunities for improvement that could be incorporated into NetInsight.

3.3.5 Document Analysis

Relevant literature, ITU recommendations, software documentation, research articles, and online technical resources were reviewed to identify internationally accepted approaches for evaluating mobile network performance.

Document analysis ensured that the system design aligned with recognized industry standards and current software development practices.

3.4 Data Gathering Results

The requirement gathering activities produced valuable information regarding user expectations and common network challenges.

The survey results of 192 respondents indicated that many respondents regularly experience slow internet speeds, unstable network connections, and inconsistent signal strength. Participants expressed interest in an application capable of providing accurate network measurements together with historical performance records..

The collected information also indicated strong demand for features such as:

- Users anonymity.
- Network speed measurement and monitoring.
- Latency monitoring.
- Display of signal strength information.
- Historical storage of previous measurements.
- User-friendly graphical interface.

- Reliable backend data storage.
- Frequency adjuster for feedback alert.

Data summary

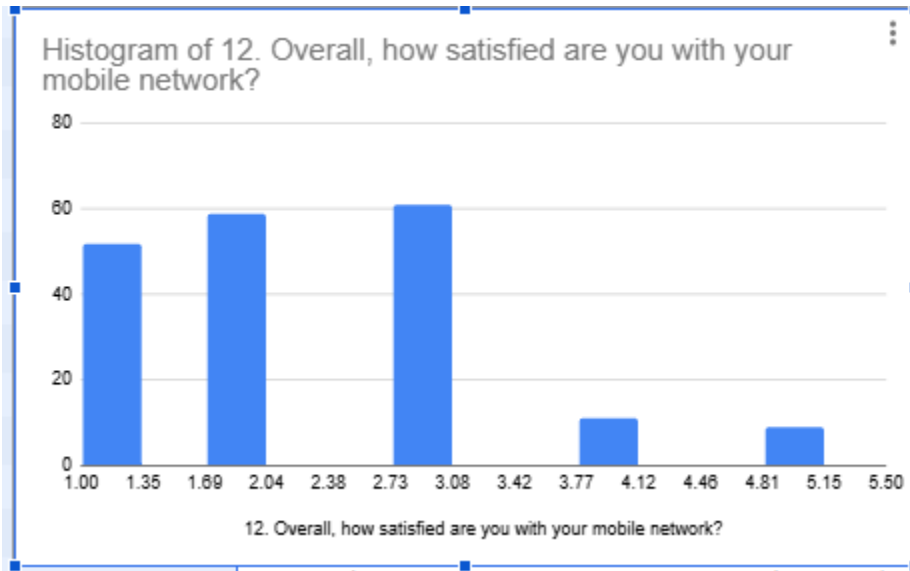


Figure 3.1: Survey Response Summary (network satisfaction)

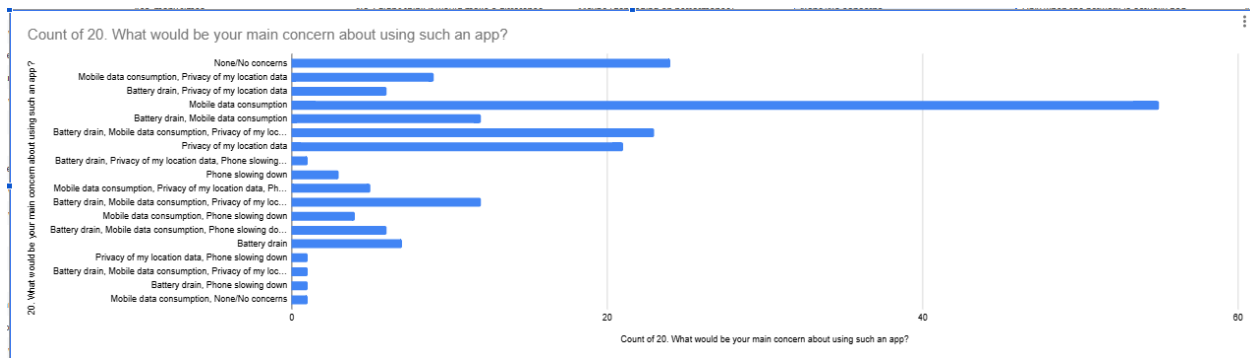


Figure 3.2: Survey response summary (concern about using NetInsight)

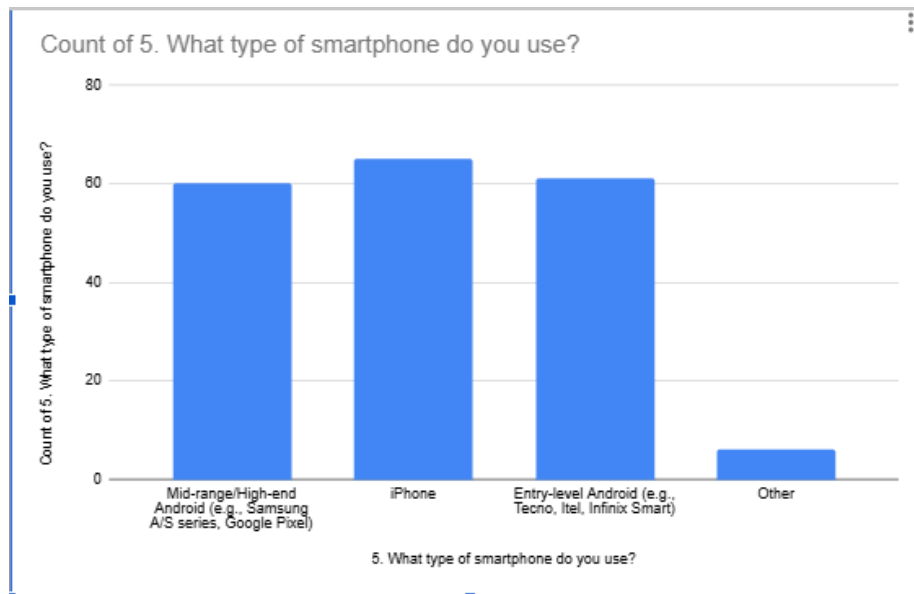


Figure 3.3: Survey response summary (user's device)

3.5 Data Cleaning

Raw survey data often contain inconsistencies, incomplete responses, duplicate entries, and irrelevant information. Data cleaning was therefore performed before analyzing the collected requirements.

The following activities were carried out during data cleaning:

- Removal of duplicate survey responses.
- Exclusion of incomplete questionnaires.
- Correction of formatting inconsistencies.
- Verification of numerical responses.
- Standardization of categorical responses.
- Elimination of responses unrelated to the project objectives.

These procedures improved the quality and reliability of the collected data, ensuring that requirement analysis was based on accurate information.

3.6 User Reluctance Assessment

Although many participants were willing to contribute, some users expressed reluctance during the requirement gathering process. Understanding these concerns helped improve data collection quality.

The major reasons for reluctance included:

- **Privacy Concerns**

Some participants were hesitant to share information because they feared their personal data might be misused. Participants were therefore informed that the survey collected only information relevant to network performance and that responses would remain confidential.

- **Limited Technical Knowledge**

Several respondents indicated that they were unfamiliar with networking terminology such as latency, packet loss, and Quality of Service. Consequently, survey questions were simplified using non-technical language wherever possible.

- **Time Constraints**

Some potential participants declined to complete the questionnaire due to limited time. To address this issue, the survey was designed to be concise while still collecting sufficient information.

- **Lack of Interest**

A small number of users believed that network performance issues were unavoidable and therefore saw little value in participating. Explaining the purpose of the project encouraged greater participation.

Overall, the majority of participants responded positively and provided useful information that significantly influenced the system requirements.

3.7 Challenges Encountered During Requirement Gathering

Several challenges were encountered during the requirement gathering process.

Firstly, obtaining responses from a sufficiently diverse group of mobile users required considerable time and effort. Secondly, users possessed varying levels of technical understanding, making it necessary to simplify certain survey questions. Finally, comparing features across existing applications required extensive review of multiple software products and technical documentation.

Despite these challenges, combining multiple requirement elicitation techniques improved the completeness and reliability of the gathered requirements.

CHAPTER FOUR

REQUIREMENT ANALYSIS

4.1 Introduction

Requirement analysis is the process of examining, refining, and organizing the information gathered during the requirement elicitation phase into a structured specification for system development. It ensures that stakeholder needs are clearly understood and translated into implementable software requirements. A thorough requirement analysis reduces ambiguity, minimizes development risks, and provides a solid foundation for system design and implementation.

This chapter analyses the requirements gathered for the NetInsight project, identifies the functional and non-functional requirements, prioritizes system features using the MoSCoW technique, and presents the Software Requirements Specification (SRS).

4.2 Analysis of Gathered Requirements

The information collected through surveys, interviews, brainstorming sessions, reverse engineering, and document analysis revealed that users require a simple, reliable, and responsive application capable of monitoring mobile network performance.

The analysis indicated that users were primarily interested in obtaining accurate measurements of their internet connection while maintaining easy access to historical network records. Users also preferred an intuitive interface that presents technical information in a clear and understandable manner.

From the developer's perspective, the system needed to be modular, scalable, secure, and maintainable. This influenced the selection of React Native for frontend development, Express.js for backend services, and MongoDB Atlas for cloud-based data storage.

The analyzed requirements were grouped into functional and non-functional categories.

4.3 Functional Requirements

Functional requirements describe the specific services and operations that the system must perform. These requirements define the expected behaviour of the NetInsight application.

The system shall provide the following functionalities:

Network Monitoring

- Measure download speed.
- Measure upload speed.

- Measure network latency.
- Display signal strength information where available.
- Display current network information.

Data Management

- Store network test results.
- Retrieve previous network performance records.
- Display historical measurements.
- Synchronize user data with the backend database.

Backend Services

- Receive requests from the mobile application.
- Validate incoming data.
- Store data in MongoDB Atlas.
- Retrieve stored information.
- Return responses through RESTful APIs.

Application Features

- Display network performance results.
- Present user-friendly dashboards.
- Allow users to view previous measurements.
- Notify users when operations complete successfully or fail.
- Prompt users for feedback.

Table 4.1: Summary of Functional Requirements

ID	Description
FR-01	The system shall run as a persistent background service that continues operating even when the application is not open by the user.

FR-05	The system shall measure and log network latency(ping) at each measurement interval
FR-16	The system shall allow user to optionally write a short free-text note describing their current network experience alongside any rating submitted.
FR-218	The system shall store all collected metric data, user ratings, and notes locally on the device in a structured database.

4.4 Non-Functional Requirements

Non-functional requirements describe the quality attributes of the system rather than its specific functionality.

- **Performance**

The application should respond quickly to user requests and retrieve stored data with minimal delay. Backend API responses should be efficient under normal operating conditions.

- **Reliability**

The system should consistently perform its intended functions without unexpected failures. Stored data should remain available whenever users access their accounts.

- **Security**

User information should be protected through secure authentication mechanisms. Communication between the frontend and backend should be handled securely, and unauthorized users should not access protected resources.

- **Usability**

The application should provide a simple and intuitive interface suitable for users with different levels of technical knowledge. Navigation between screens should be straightforward and consistent.

- **Maintainability**

The software architecture should be modular to simplify debugging, maintenance, and future feature additions.

- **Scalability**

The backend should support future expansion, allowing additional features and increased numbers of users without requiring major architectural changes.

- **Compatibility**

The application is designed primarily for Android devices using React Native and Expo. The backend APIs should remain platform-independent for possible future expansion.

- **Performance**

Table 4.2: Summary of Non-Functional Requirements

ID	Description
NFR-01	The system's background service shall consume no more than 5% of total device battery per hour during normal monitoring operation.
NFR-05	A first-time user shall be able to install the application, complete the onboarding process, and understand the main dashboard without any external guidance or training.
NFR-10	The system shall not lose any locally stored data in the event of an application crash or unexpected device restart.
NFR-14	No personally identifiable information including name, phone number, or device IMEI shall be collected, stored or transmitted by the application.

4.5 Requirement Prioritisation Using the MoSCoW Technique

Since not every feature could be implemented within the available project timeline, the requirements were prioritised using the MoSCoW method.

The MoSCoW technique classifies requirements into four categories:

- Must Have
- Should Have
- Could Have
- Won't Have (for this version)

Must Have

These requirements are essential for the successful operation of the application.

- Minimal Data consumption.
- Minimal Battery Impact.
- Privacy & Approximate Location.

- Reliability for “No signal” scenarios.

Should Have

These requirements significantly improve the user experience but are not essential for basic functionality.

- Improved network statistics.
- Better visualization of historical results.
- Enhanced error handling.

Could Have

These requirements provide additional value if sufficient development time is available.

- Dark mode.
- Push notifications.
- Exporting network reports.
- Interactive charts and graphs.
- Location-based performance comparison.

Won't Have (Current Version)

The following features were considered beyond the scope of the current project.

- Artificial intelligence for network prediction.
- Direct communication with mobile network operators.
- Automatic network optimisation.
- Offline synchronization.
- Map coverage

4.6 Software Requirements Specification (SRS)

The Software Requirements Specification provides a structured description of the system's expected behaviour.

4.6.1 Purpose

The purpose of NetInsight is to provide Android users with a platform for monitoring mobile network Quality of Service while supporting the collection and storage of network performance information.

4.6.2 Intended Users

The application is intended for:

- Smartphone users.
- Students.
- Researchers.
- Mobile network subscribers.
- Developers evaluating network performance.

4.6.3 System Overview

NetInsight consists of three major components:

- Android mobile application developed using React Native and Expo.
- Backend server developed using Express.js.
- MongoDB Atlas cloud database accessed through Mongoose.

These components communicate through RESTful APIs.

4.6.4 Functional Specification

The application shall:

- Ensure users anonymity.
- Measure selected network performance metrics.
- Store network measurements.
- Retrieve historical measurements.
- Display collected information.
- Communicate with the backend through REST APIs.

4.6.5 External Interfaces

User Interface

The application provides graphical interfaces for:

- Splash screen
- Home dashboard.
- History.
- Settings

Hardware Interface

The application requires:

- Android smartphone.
- iOS devices
- Cellular or Wi-Fi network.

Software Interface

The application interacts with:

- React Native.
- Expo.
- Node.js.
- Express.js.
- MongoDB Atlas.
- Mongoose.
- REST APIs.

4.6.6 Constraints

The development of NetInsight is subject to several constraints:

- Android platform only.
- Dependence on third-party packages.
- Limited access to large-scale network testing infrastructure.

4.6.7 Assumptions

The following assumptions were made during development:

- Users possess Android smartphones.
- MongoDB Atlas services remain available.
- Backend APIs remain operational.

4.7 Requirement Traceability

Requirement traceability ensures that every identified requirement can be linked to its implementation during system development.

The traceability matrix links stakeholder requirements to corresponding system functionalities, design components, implementation modules, and testing activities. This improves project management and simplifies future maintenance.

CHAPTER FIVE

SYSTEM MODELLING AND DESIGN

5.1 Introduction

Following the analysis of system requirements, the next stage in the software development process is system modelling and design. System design provides a blueprint that guides the implementation of the application by describing how different system components interact to achieve the specified requirements. It transforms user requirements into logical and physical representations that developers can implement.

For the NetInsight project, several modelling techniques were used to describe the structure, behaviour, and deployment of the application. These models help stakeholders understand the operation of the system before implementation while reducing ambiguity during development.

This chapter presents the system design of NetInsight through various software engineering models, including the Context Diagram, Data Flow Diagram (DFD), Use Case Diagram, Sequence Diagrams, Class Diagram, and Deployment Diagram.

5.2 System Architecture

NetInsight adopts a three-tier architecture consisting of the presentation layer, application layer, and data layer.

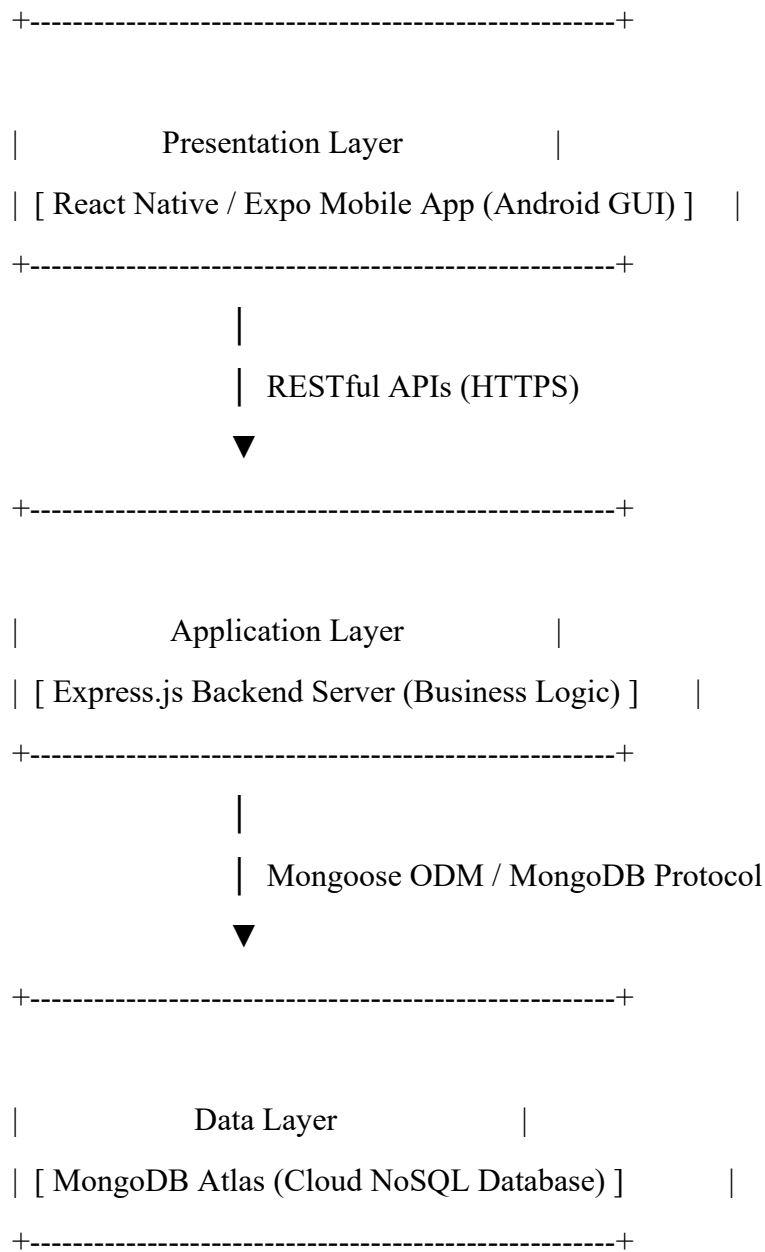
The presentation layer consists of the Android mobile application developed using React Native and Expo. It provides the graphical user interface through which users interact with the system.

The application layer consists of an Express.js backend server that processes requests from the mobile application, performs business logic, validates incoming data, and communicates with the database.

The data layer consists of MongoDB Atlas, a cloud-based NoSQL database responsible for storing user accounts, network performance records, and other application data.⁸

The three layers communicate using RESTful APIs, ensuring a clear separation of responsibilities and making the application easier to maintain and extend.

Figure 5.1: Overall System Architecture



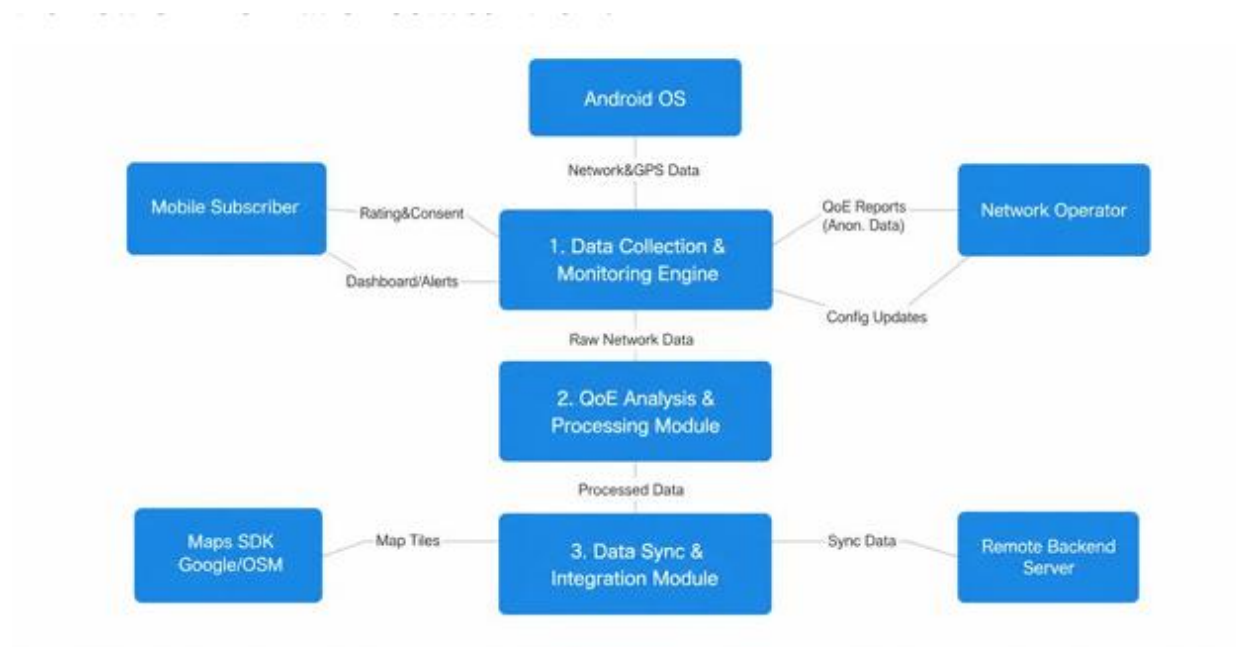
5.3 Context Diagram

A Context Diagram provides a high-level view of the system by illustrating its interaction with external entities. It treats the entire software application as a single process while showing the flow of information between the system and its users or external services.

For NetInsight, the primary external entity is the mobile user, who interacts with the application by registering, logging in, performing network tests, viewing results, and accessing historical records. The system also communicates with MongoDB Atlas through the backend for data storage and retrieval.

The Context Diagram helps define the system boundary and identifies the external actors involved in system operation.

Figure 5.2: Context Diagram



5.4 Data Flow Diagram (DFD)

A Data Flow Diagram illustrates how information flows within a system. It identifies processes, data stores, external entities, and data movement between them.

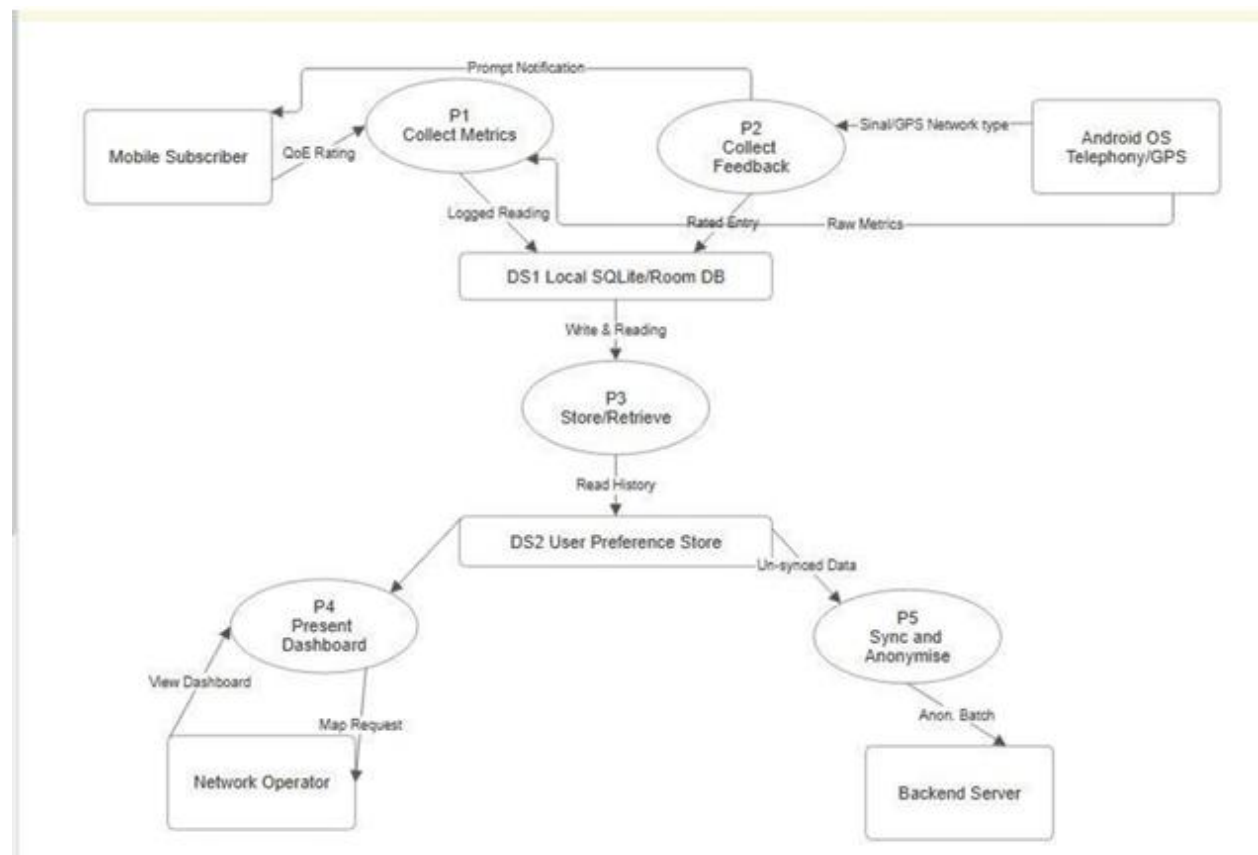
5.4.1 Level 0 Data Flow Diagram

The Level 0 DFD provides an overview of the major processes performed by NetInsight. These include:

- Running speed tests.
- Reporting issues.
- Viewing coverage maps.
- Comparing operators.

The diagram demonstrates how user requests pass through the backend before interacting with the database.

Figure 5.3: Level 0 Data Flow Diagram



5.4.2 Level 1 Data Flow Diagram

The Level 1 DFD expands the major processes identified in the Level 0 diagram by showing their internal activities.

Input

For the input data collected by the systems are from:

- User device data.
- User Reports.

Output:

Raw network data stored in database

Process 2.0: Analyze Network Performance

The system collected data to:

- Identify weak signal areas
- Compares operators
- Detect poor network quality

Output:

- Network analytics
- Performance statistics

Process 3.0: Generate coverage Maps

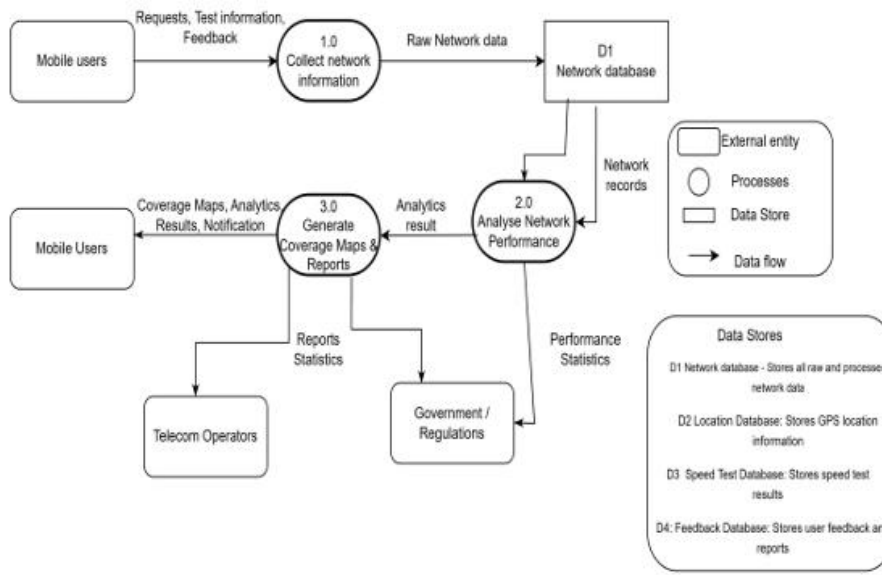
The system uses the data and analyzes it to create:

- Coverage maps
- Signal heat maps
- Operator comparison visuals

Output:

Provide Visuals reports to users and operators

Figure 5.4: Level 1 Data Flow Diagram



5.5 Use Case Diagram

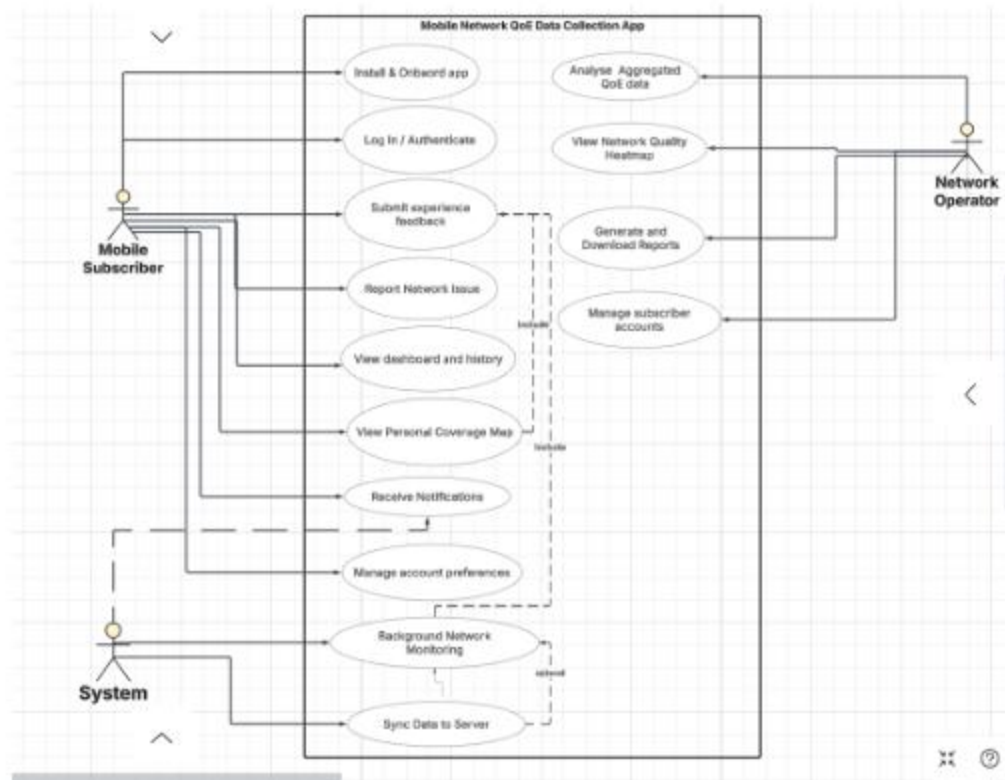
The Use Case Diagram describes the interactions between users and the NetInsight application. It identifies the major services offered by the system from the user's perspective.

The primary actor is the registered user, who can perform several actions including:

- View the dashboard.
- View previous network measurements.
- Log out.

The Use Case Diagram provides a simplified representation of the system's functional requirements.

Figure 5.5: Use Case Diagram



5.6 Sequence Diagrams

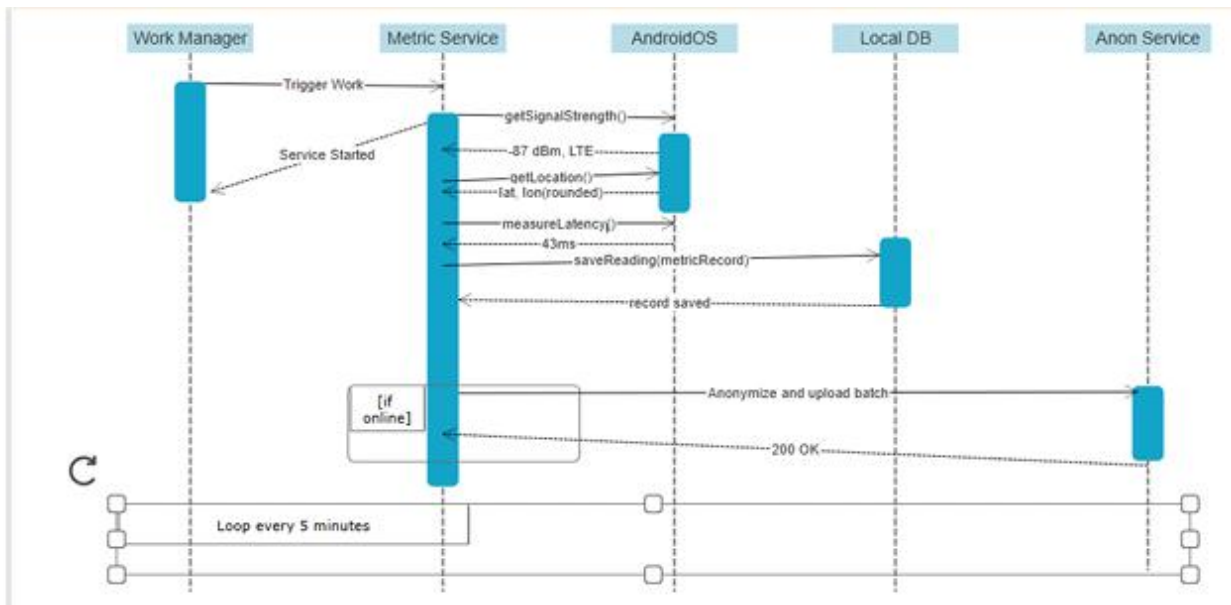
Sequence diagrams illustrate the chronological order in which system components exchange messages during the execution of specific operations.

5.6.1 User Registration Sequence Diagram

This sequence diagram illustrates the registration process.

The process begins when a new user submits registration information through the mobile application. The frontend sends the data to the Express backend, which validates the information before storing it in MongoDB Atlas. A success or failure response is then returned to the application.

Figure 5.6: Sequence Diagram



5.7 Class Diagram

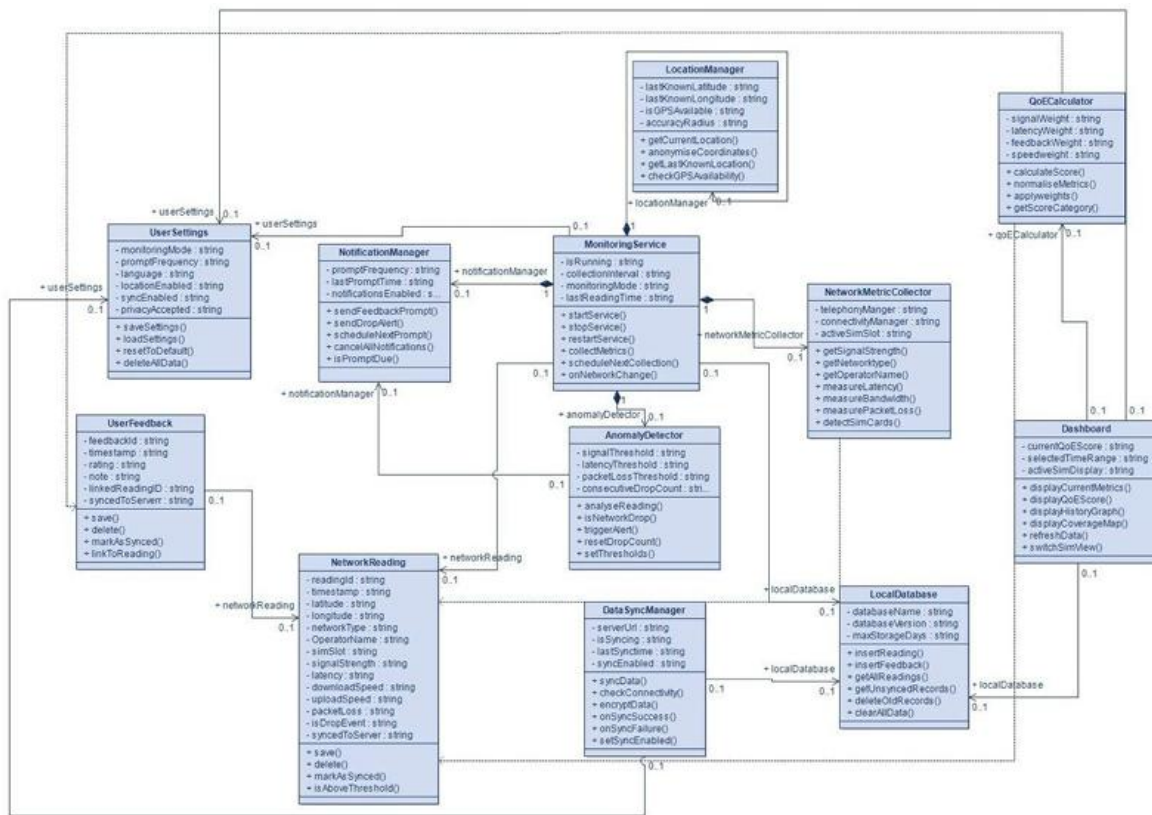
The Class Diagram represents the static structure of the NetInsight system. It identifies the major classes, their attributes, methods, and relationships.

Typical classes in the system include:

- Network reading
- Monitoring service
- Network metric collector
- Location manager

Each class performs specific responsibilities that collectively support the application's functionality.

Figure 5.9: Class Diagram



5.8 Deployment Diagram

The Deployment Diagram illustrates the physical arrangement of hardware and software components required to operate NetInsight.

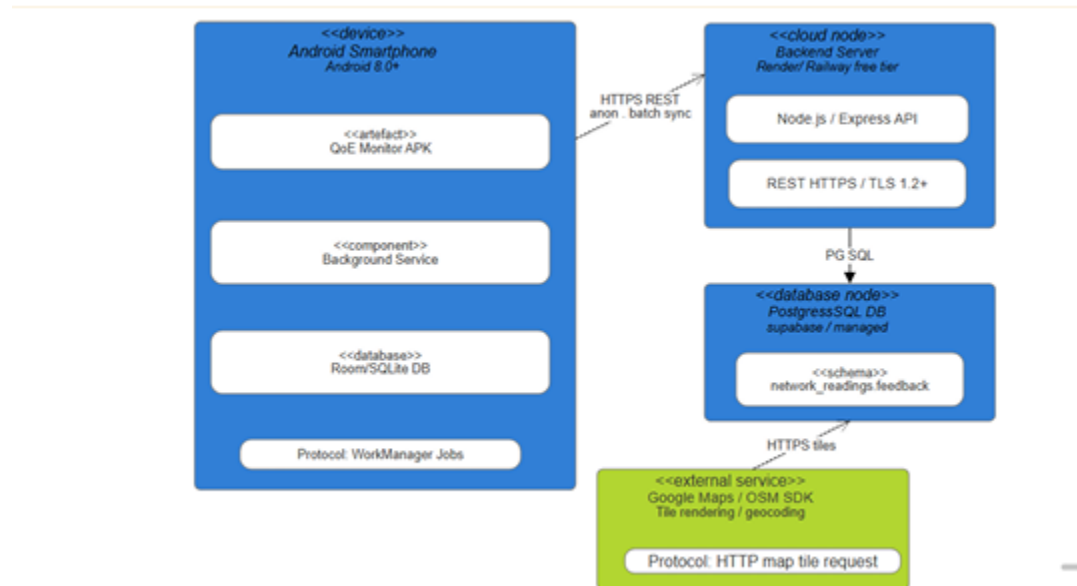
The deployment consists of:

- Android smartphone running the React Native application.
- Internet connection.
- Express.js backend server.
- MongoDB Atlas cloud database.

Communication between the mobile application and backend occurs through HTTP requests, while the backend interacts securely with MongoDB Atlas.

The deployment architecture allows the application to be scalable and accessible from any Android device with internet connectivity.

Figure 5.10: Deployment Diagram



5.9 Design Considerations

Several software engineering principles guided the design of NetInsight.

- **Modularity**

The application is divided into independent modules, simplifying development, testing, and maintenance.

- **Scalability**

The backend architecture allows future addition of features without requiring significant redesign.

- **Maintainability**

The separation between frontend, backend, and database improves code organization and simplifies future updates.

- **Security**

User authentication mechanisms restrict unauthorized access to protected resources.

- **Usability**

The application interface was designed to be intuitive, ensuring users can easily navigate between screens and access network information.

CHAPTER SIX

USER INTERFACE DESIGN AND IMPLEMENTATION

6.1 Introduction

The user interface (UI) is one of the most important aspects of a mobile application because it determines how users interact with the system. An effective interface should be visually appealing, intuitive, responsive, and easy to navigate. Good UI design improves user satisfaction by reducing the learning curve and enabling users to accomplish tasks efficiently.

For the NetInsight project, emphasis was placed on developing a clean and user-friendly interface suitable for Android devices. The application was designed using Figma before implementation in React Native with Expo. The frontend communicates with the Express.js backend through RESTful APIs, allowing users to access network monitoring features and stored data.

This chapter presents the application's identity, design considerations, user interface mockups, frontend implementation, and screenshots of the developed application.

6.2 Application Identity

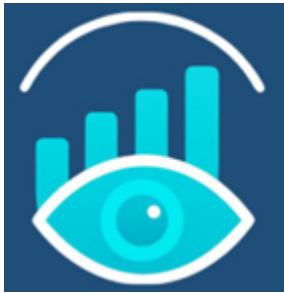
The application developed for this project is named NetInsight.

The name combines the words "Network" and "Insight", reflecting the application's primary objective of helping users understand the performance of their mobile network through meaningful measurements and visualized information.

6.2.1 Application Logo

The NetInsight logo was designed to represent mobile connectivity, communication, and network analysis. The design incorporates modern visual elements that reflect internet connectivity and data transmission while maintaining simplicity for easy recognition.

Figure 6.1: NetInsight Application Logo



6.2.2 Colour Scheme

The application's colour scheme was selected to provide a professional appearance while ensuring readability and consistency throughout the interface.

The primary colour is blue, representing trust, reliability, communication, and technology. Blue is widely used in networking and information technology applications because it conveys stability and professionalism.

Secondary colours such as white and light grey were used to improve readability and create a clean interface. Accent colours were applied where necessary to highlight important buttons, notifications, and interactive components.

6.2.3 Typography

Readable typography is essential for ensuring that users can easily understand information displayed within the application.

The interface uses modern sans-serif fonts that provide excellent readability across different Android screen sizes. Appropriate font sizes, spacing, and text hierarchy were maintained to distinguish headings, labels, buttons, and body text.

Consistency in typography contributes to a professional appearance and improves the overall user experience.

6.3 User Interface Design

Before implementation, the application interfaces were designed using Figma. The design process focused on creating a simple navigation structure while minimizing unnecessary complexity.

The following principles guided the interface design:

- Simplicity.
- Consistency.
- Readability.
- Responsiveness.
- Accessibility.
- Minimal user effort.
- Clear navigation.

The interfaces were reviewed and refined before implementation to ensure they aligned with the identified user requirements.

6.3.1 Figma Design Mockups

The Figma prototype served as the visual blueprint for frontend development. It illustrated the layout of application screens, navigation flow, colour usage, typography, and component placement.

Figure 6.2: Figma Splash Screen Mockup

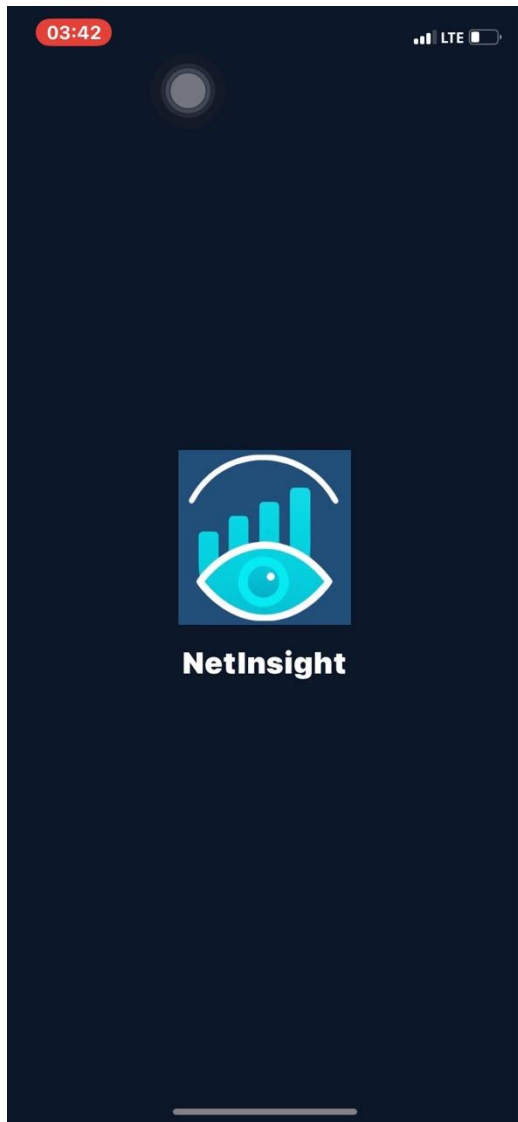


Figure 6.3: Figma Language Screen Mockup

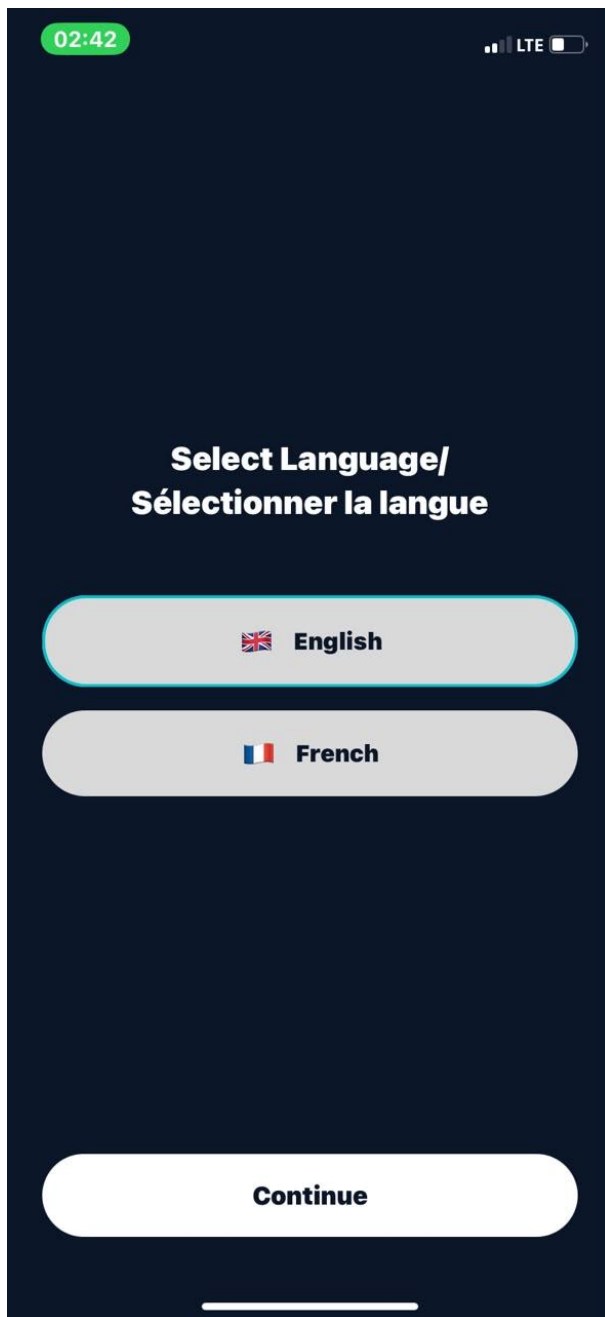
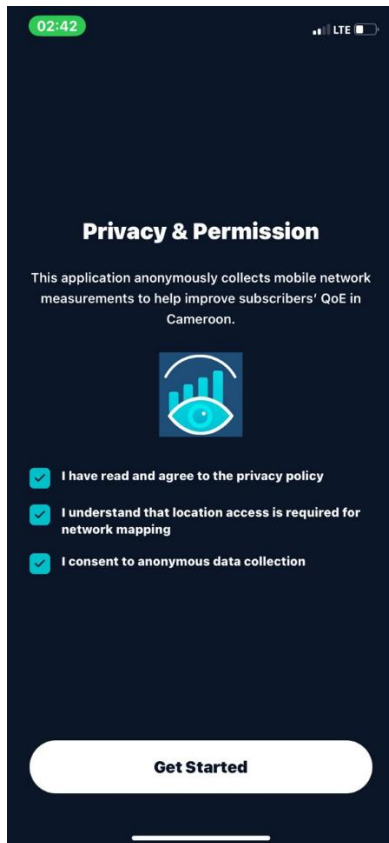


Figure 6.4: Figma Privacy Screen Mockup



6.4 Frontend Implementation

The frontend of NetInsight was implemented using React Native with Expo. This development framework enables the creation of modern Android applications using JavaScript and reusable components.

React Native was selected because it provides efficient UI development, supports component-based architecture, and simplifies future maintenance. Expo further accelerated development by providing built-in tools for testing, debugging, and deployment.

The frontend communicates with the backend through RESTful API requests, allowing users to authenticate, retrieve data, and store network measurements.

6.5 Description of Implemented Screens

The application consists of several interfaces, each designed to perform a specific function.

6.5.1 Dashboard Metric Screen

This screen displays all network metrics on sub-cards for users to see and a constant flashing red light to signal monitoring.

The design emphasizes simplicity while presenting the NetInsight branding clearly.



6.6 Challenges Encountered During Implementation

Several challenges were encountered during frontend implementation.

One challenge involved ensuring reliable communication between the React Native application and the Express.js backend. This required careful configuration of API endpoints and request handling.

Another challenge involved maintaining consistent user interface behaviour across different Android devices and screen sizes. Responsive layouts and flexible component positioning were adopted to improve compatibility.

Managing application state and handling asynchronous API requests also required careful implementation to ensure smooth user interactions and appropriate error handling.

Despite these challenges, the application successfully integrates its frontend with the backend database and provides a functional user interface for monitoring network performance.

CHAPTER SEVEN

DATABASE DESIGN AND IMPLEMENTATION

7.1 Introduction

A well-designed database is essential for ensuring efficient data storage, retrieval, and management in any software application. It provides a structured means of organizing information while maintaining data integrity, consistency, and security. In modern web and mobile applications, the database serves as the central repository through which application data is accessed and managed.

For the NetInsight project, MongoDB Atlas was selected as the database management system because of its flexibility, scalability, and compatibility with JavaScript-based technologies. The database is accessed through Mongoose, an Object Data Modeling (ODM) library for Node.js, while the Express.js backend provides RESTful APIs that allow the React Native frontend to interact with the stored data.

This chapter discusses the data elements identified for the system, the conceptual database design, the Entity Relationship (ER) diagram, the implementation of the MongoDB database, the backend API implementation, and the integration of the database with both the backend and frontend.

7.2 Data Elements

Data elements represent the individual pieces of information stored and managed by the application. These elements were identified during the requirement gathering and analysis phases and were used to design the database structure.

The major data entities in NetInsight include:

Network Metric data

Collects metric automatically and continuously at periodic intervals

Typical attributes include:

- Device Id
- Timestamp
- Connection type
- Downloadspeed
- Uploadspeed

User feedback

Captures user's subjective feedback.

Typical attributes include:

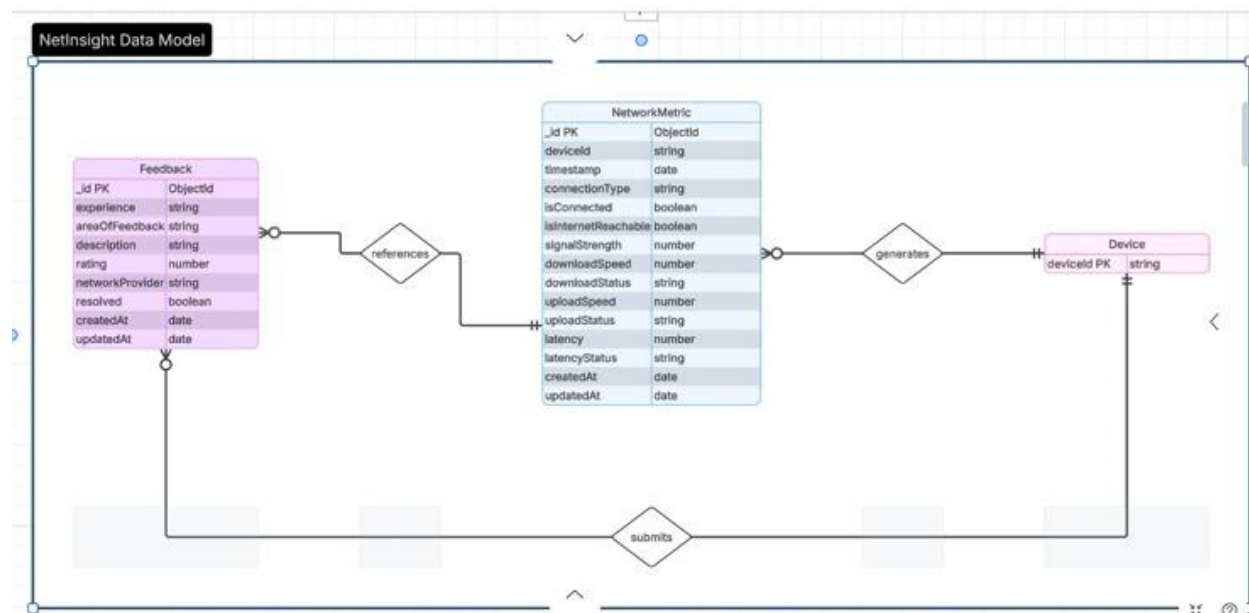
- experience
- areaoffeedback
- description
- rating

7.3 Conceptual Database Design

The conceptual database design identifies the major entities within the system and the relationships between them without considering implementation details.

The conceptual model provides a high-level understanding of how application data is organized before translating it into database collections.

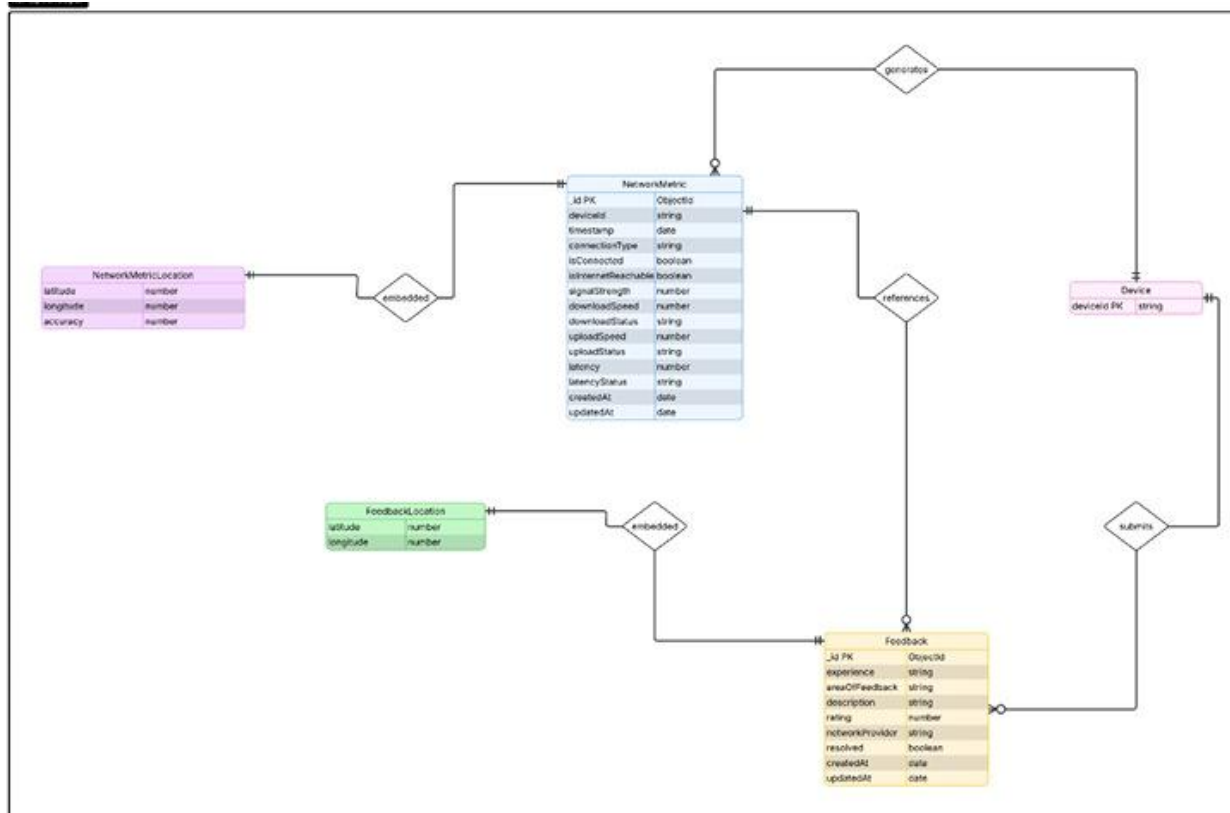
Figure 7.1: Conceptual Database Design



7.4 Entity Relationship Diagram

The Entity Relationship (ER) diagram provides a graphical representation of the entities, their attributes, and the relationships that exist between them in NetInsight.

Figure 7.2: Entity Relationship Diagram



7.5 Database Implementation

MongoDB Atlas was selected as the database platform because it provides cloud-based storage, high availability, automatic backups, and easy integration with Node.js applications.

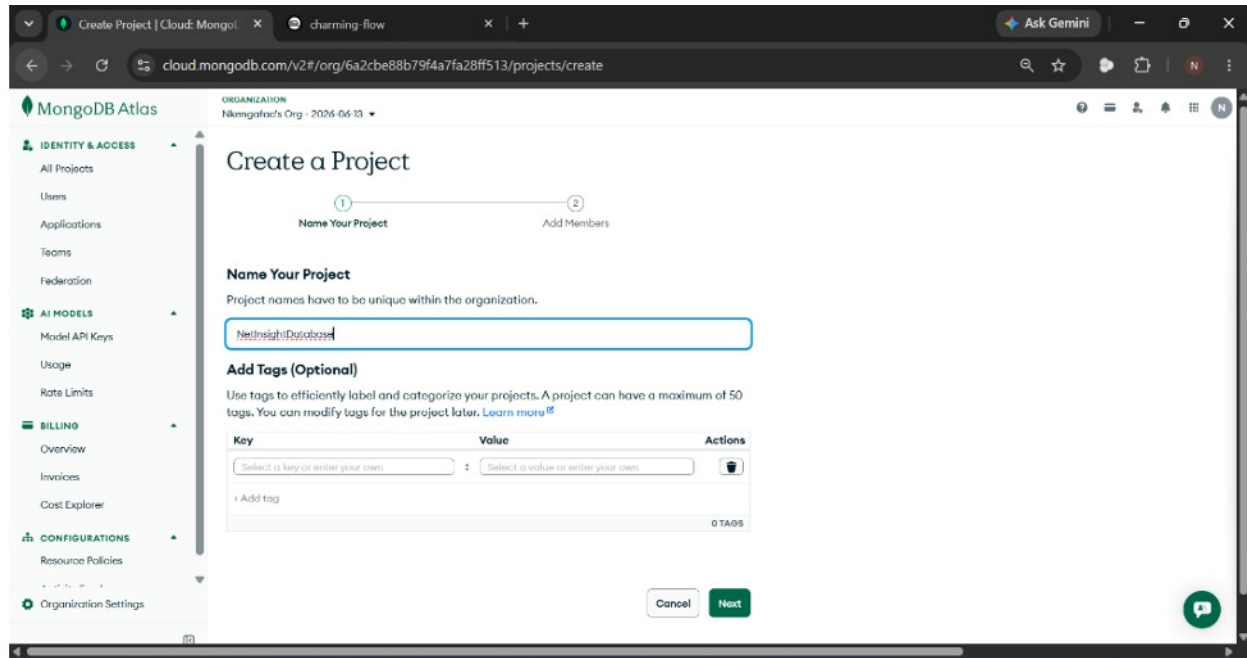
Unlike traditional relational databases, MongoDB stores information as JSON-like documents within collections. This document-oriented approach simplifies the storage of application data and supports future expansion.

The primary collections implemented in NetInsight include:

- Network metrics Collection
- Feedback

These collections are accessed using Mongoose models, which enforce schema validation and simplify database operations.

Figure 7.3: MongoDB Atlas Database



7.6 Mongoose Schemas

Mongoose provides schema definitions that specify the structure of documents stored in MongoDB.

Separate schemas were created for each major entity in the application.

The Network metric schema defines speed, timestamp, etc.

The feedback schema defines fields for storing measured timestamp, experience, rating, description.

Schema validation ensures that incomplete or incorrectly formatted information is rejected before being stored in the database.

Figure 7.4: Mongoose Schema Implementation

```
models > JS NetworkMetric.js > ...
1  const mongoose = require('mongoose');
2
3  const locationSchema = new mongoose.Schema(
4  {
5    latitude: { type: Number, default: 0 },
6    longitude: { type: Number, default: 0 },
7    accuracy: { type: Number, default: 0 },
8  },
9  { _id: false }
10 );
11
12 const networkMetricSchema = new mongoose.Schema(
13 {
14   deviceId: { type: String, required: true },
15   timestamp: { type: Date, required: true, default: Date.now },
16   connectionType: { type: String, default: 'unknown' },
17   isConnected: { type: Boolean, default: false },
18   isInternetReachable: { type: Boolean, default: false },
19   signalStrength: { type: Number, default: -100 },
20   downloadSpeed: { type: Number, default: 0 },
21   downloadStatus: { type: String, default: 'unknown' },
22   uploadSpeed: { type: Number, default: 0 },
23   uploadStatus: { type: String, default: 'unknown' },
24   latency: { type: Number, default: 0 },
25   latencyStatus: { type: String, default: 'unknown' },
26   location: { type: locationSchema, default: () => ({}) },
27 },
28 { timestamps: true }
29 );
```

```
models > JS Feedback.js > ...
1  const mongoose = require('mongoose');
2
3  const feedbackLocationSchema = new mongoose.Schema(
4  {
5    latitude: { type: Number, default: 0 },
6    longitude: { type: Number, default: 0 },
7  },
8  { _id: false }
9  );
10 const feedbackSchema = new mongoose.Schema(
11 {
12   experience: {
13     type: String,
14     enum: ['very_poor', 'poor', 'fair', 'good', 'excellent'],
15     required: true,
16   },
17   areaOfFeedback: { type: String, required: true },
18   description: { type: String, default: '' },
19   rating: { type: Number, min: 1, max: 5, required: true },
20   location: { type: feedbackLocationSchema, default: () => ({}) },
21   networkProvider: { type: String, default: 'Anonymous' },
22   resolved: { type: Boolean, default: false },
23 },
24 { timestamps: true }
25 );
26
27 feedbackSchema.index({ createdAt: -1 });
28
29 module.exports = mongoose.model('Feedback', feedbackSchema);
```

7.7 Backend Implementation

The backend of NetInsight was developed using Express.js running on Node.js. It acts as the intermediary between the React Native frontend and the MongoDB database.

Its primary responsibilities include:

- Validating input data.
- Executing database operations.
- Returning appropriate HTTP responses.

The backend follows a RESTful architecture, allowing communication through standard HTTP methods such as GET, POST, PUT, and DELETE where applicable.

7.8 API Endpoints

Several REST API endpoints were implemented to support communication between the frontend and backend.

Examples include endpoints for:

- Feedback log in
- Network metrics
- Server.js

Table 7.2: API Endpoints

Insert Table 7.2 here showing Endpoint, HTTP Method, Purpose, and Expected Response.

Method	Path	Status	Description
POST	/api/feedback	201 created	Validates and saves user feedback entry. Returns 400 if experience, areaoffeedback, or rating is invalid
GET	/api/network-metrics/stats	200 OK	Returns today's aggregate averages computed via MongoDB aggregation pipeline
GET	/api/network-metrics	200 ok	Returns recent samples, optionally filtered by deviceId, and limited by limit.

7.9 Connecting the Database to the Backend and Frontend

The integration process involved connecting MongoDB Atlas to the Express.js backend using Mongoose. After establishing a successful database connection, RESTful API endpoints were created to perform Create, Read, Update, and other required operations.

The React Native frontend communicates with these APIs using HTTP requests. When a user registers, logs in, or performs a network test, the frontend sends the required information to the backend. The backend validates the request, interacts with MongoDB Atlas, and returns an appropriate response to the application.

This architecture ensures a clear separation of responsibilities between the user interface, business logic, and data storage layers.

Figure 7.5: Backend–Database Connection

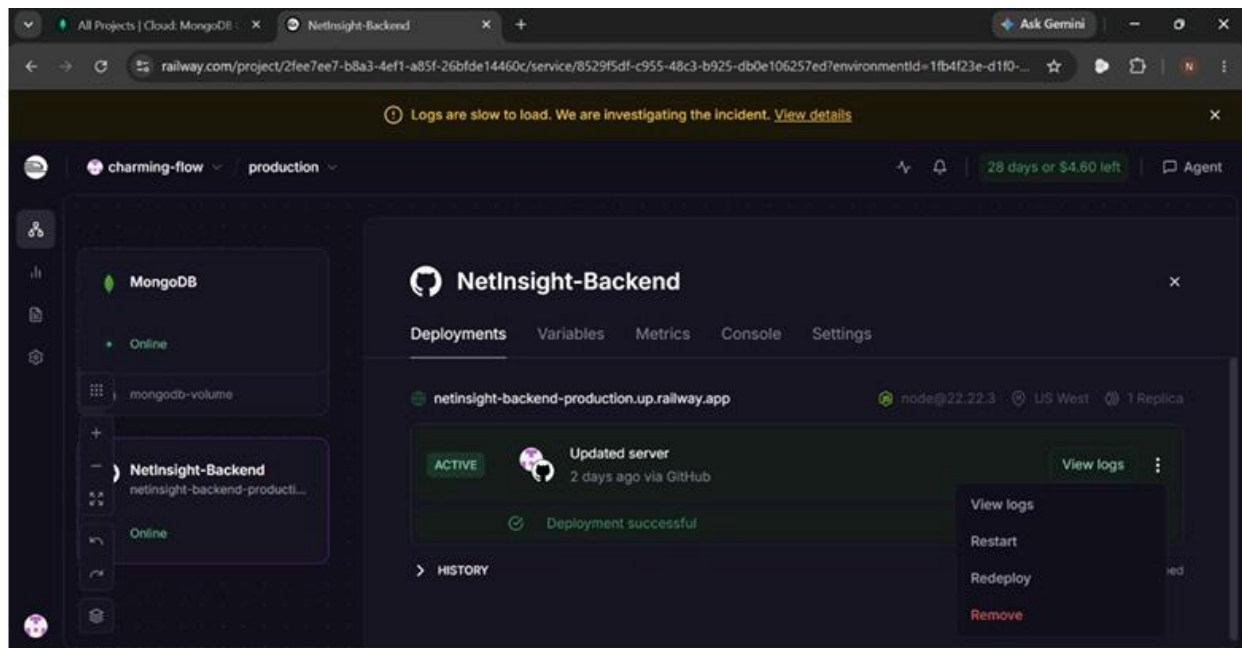
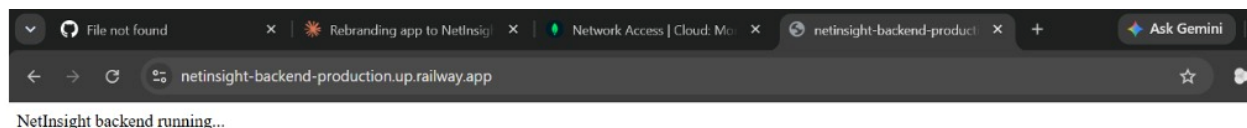


Figure 7.6: Frontend–Backend Communication



7.10 Challenges Encountered

During database implementation, several challenges were encountered, including configuring secure connections to MongoDB Atlas, validating incoming data, handling API errors, and ensuring consistent communication between the frontend and backend.

These challenges were addressed through proper schema validation, structured API design, testing of endpoints, and careful configuration of database connection settings.

CHAPTER EIGHT

TESTING

8.1 Introduction

Software testing is a critical phase of the Software Development Life Cycle (SDLC) that ensures a system functions correctly, meets user requirements, and performs reliably under expected operating conditions. The primary objective of testing is to identify defects before deployment, verify that implemented features work as intended, and improve the overall quality of the software.

For the NetInsight project, testing was conducted throughout the development process to verify the functionality of both the frontend and backend components. Different testing approaches were employed to ensure that user interactions, API communication, and database operations worked correctly. This chapter discusses the testing strategy adopted, the different types of testing performed, the results obtained, and the findings from the testing activities.

8.2 Testing Strategy

A systematic testing strategy was adopted to ensure that the application met its functional and non-functional requirements. Testing was performed incrementally as different modules were developed and integrated.

The strategy consisted of the following stages:

- Testing individual components independently.
- Testing communication between system modules.
- Testing the complete application as an integrated system.
- Validating the application through user interaction.

This approach helped identify defects early in the development process and reduced the likelihood of errors propagating into later stages of implementation.

8.3 Test Environment

Testing was carried out using the software and hardware environment employed during development.

The test environment consisted of:

- Android smartphone or Android emulator.
- React Native application running through Expo.
- Node.js runtime environment.
- Express.js backend server.
- MongoDB Atlas cloud database.
- Stable internet connection.

This environment closely resembled the intended deployment environment, ensuring realistic testing conditions.

REFERENCES

- Alexander, J., & Mark, R. (2021). Identifying drivers of user reluctance in digital systems. *Journal of Behavioral Information Systems*. Cited by: 1
- Bhattacharya, S. (2023). Agile software development: Overcoming the stakeholder engagement gap. *Journal of Software Engineering and Applications*, 16(4), 112–128.
- Bourque, P., & Fairley, R. E. (2014). *Guide to the Software Engineering Body of Knowledge (SWEBOK Guide)*. IEEE Computer Society.
- Brooks, F. P. (1987). No silver bullet: Essence and accidents of software engineering. *Computer*, 20(4), 10–19.
- Browne, G. J., & Ramesh, V. (2002). Psychological connections in requirements elicitation: An evaluation of selected techniques. *IEEE Transactions on Software Engineering*, 28(6), 586–596.
- Costain, G., & McKenna, S. (2011). The hierarchy of requirements elicitation challenges. *Information Systems Journal*, 21(5), 450–472.
- Daniali, H., & Flaten, M. A. (2021). What psychological factors make individuals believe they are infected by Coronavirus 2019? *Frontiers in Psychology*, 12, 667722. <https://doi.org/10.3389/fpsyg.2021.667722> Cited by: 16
- Ezell, J. D., Lending, D., & Dillon, S. (2019). Improving student requirements elicitation skills: An action research study. *Journal of Information Systems Education*, 30(1), 45–58.
- Gaikwad, S., & Joeg, P. (2017). Human factors in requirements engineering: A study on non technical user participation. *International Journal of Computer Applications*, 165(8), 14–20.
- Hinderks, A., et al. (2022). User experience in agile development: A systematic literature review. *Information and Software Technology*, 144, 106791.
- Human Factors and Ergonomics Society. (2021). Human Readiness Level (HRL) scale in the system development process. HFES Standards Committee.
- Inayat, I., et al. (2015). A systematic literature review on agile requirements engineering practices and challenges. *Computers in Human Behavior*, 51, 915–929.
- Kamthan, P., & Shahmir, N. (2019). On the role of empathy in requirements engineering. *IEEE 27th International Requirements Engineering Conference (RE)*, 363–368.
- Kaur, P., et al. (2023). Barriers to AI adoption in healthcare: A Middle Eastern perspective. *International Journal of Intelligent Systems*, 38(1), 45–62.
- Konisky, D. M., Ansolabehere, S., & Carley, S. (2020). To support or resist? The role of place-based identity in stakeholder engagement. *Academy of Management Perspectives*.
- Macaulay, L. A. (1996). *Requirements Engineering*. Springer-Verlag.
- Martins, L. E. G., et al. (2019). Challenges in eliciting requirements from busy stakeholders: A qualitative study. *Software Quality Journal*, 27(3), 1105–1132.
- Ozkaya, M., Akdur, D., Toptani, E. C., Kocak, B., & Kardas, G. (2023). Practitioners' Perspectives towards Requirements Engineering: A Survey. *Systems*, 11(2), 65. <https://doi.org/10.3390/systems11020065> Cited by: 19
- Project Management Institute. (2016). Overcoming stakeholder resistance through dialogue. PMI White Paper Series.